

# Task Recursive Tree Agent

让机器人从“能执行”走向“可解释、可恢复、可扩展”

一套把自然语言意图、任务树、规划工件、物理动作、观测验证和故障修复串成闭环的 Agent 运行时。

1

权威任务树状态

3

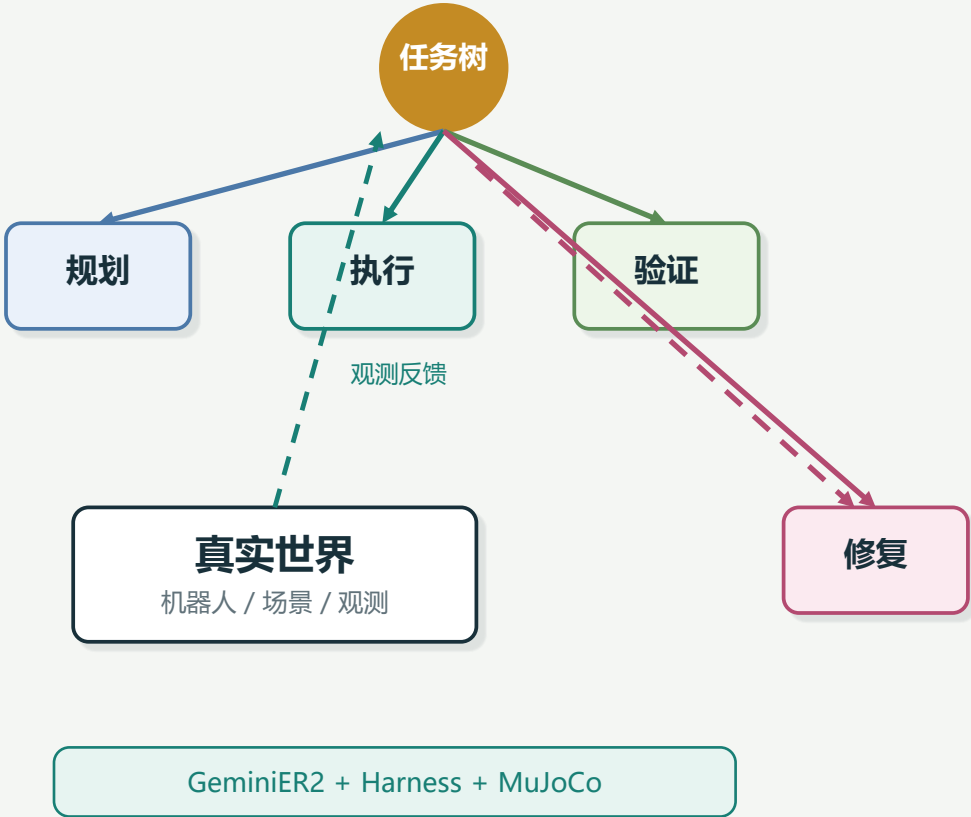
节点职责类型

2

状态边界：任务 / 世界

失败不是异常分支，而是可追踪、可验证、可重试的任务。

从意图到现实



# 这是一个把 Agent 决策变成可治理执行系统的架构

老板先看结果，CTO 再看实现：同一套设计同时回答可靠性、解释性、扩展性和安全性。

**01**

## 决策与动作分离

LLM 只提交受约束的任务意图；机器人动作必须经过任务树和物理边界。

**02**

## 任务树成为控制平面

一个 Kernel 统一处理生命周期、顺序、重试、暂停、取消和修复。

**03**

## 状态和工件可追溯

世界快照、规划工件、事件日志和执行栈让每一步都有证据。

**04**

## 失败显式递归

路线被挡不是“再试一次”，而是挂载 repair 子树，完成修复后回到原节点。

## 一条任务如何穿过系统



**核心判断：把“模型可能做什么”与“系统允许现在做什么”分开，才能把 Agent 带进真实世界。**

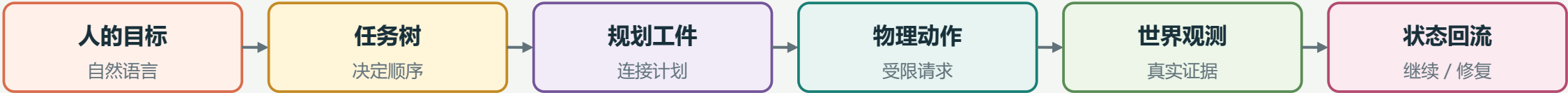
# 先把三个对象分开，整套架构就容易理解了

可以把系统类比成：任务清单 + 现场账本 + 带版本的施工图。三者各自回答一个问题，不能混成一个大上下文。



术语速译：TaskProgram = 任务意图 · GraphDelta = 增量挂载 · Diagnostic = 结构化失败 · Repair = 修复子树

## 一条任务如何把三者串起来



一句话：树管顺序，世界给证据，工件把计划接到动作；三者分开，系统才可审计。

# 为什么不能让 LLM 直接调用机器人 API?

直接调用在 Demo 阶段很快，但一旦进入连续场景、动态障碍和故障恢复，隐式状态会变成系统性风险。

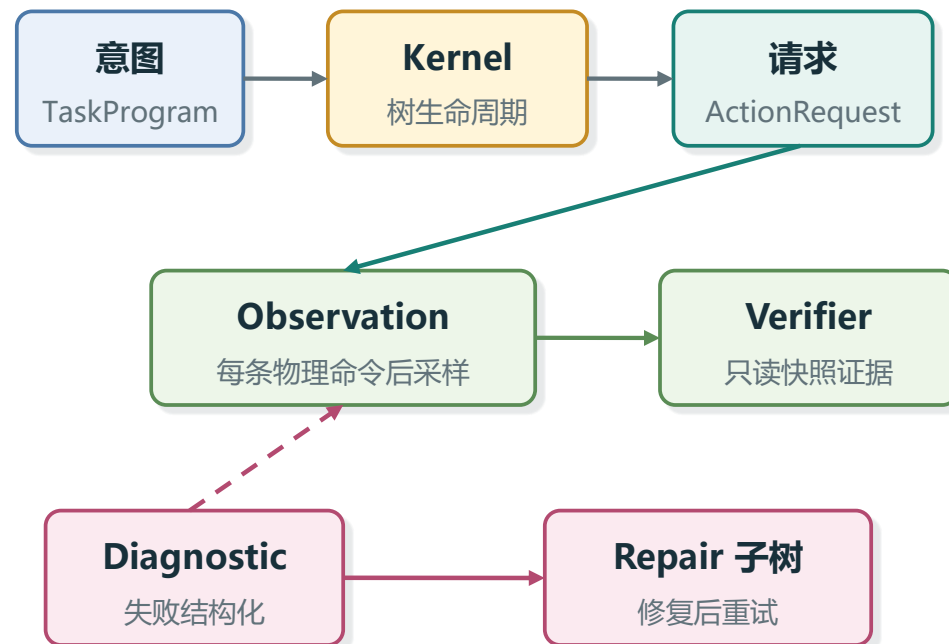
## 传统黑盒 Agent



- 计划藏在模型上下文里，外部无法审计
- 工具执行后，模型未必知道真实世界发生了什么
- 重试可能重复副作用，恢复逻辑散落在各个工具里
- 计划一旦过期，系统没有统一的失效判定

风险：不可解释 · 难恢复 · 难扩展

## Task Recursive Tree Agent



收益：每一步都有“谁决定、用什么工件、执行了什么、观察到了什么、失败如何处理”的答案。

# 六条原则，决定这套系统为什么可控

这些原则不是代码风格，而是所有权、状态、数据契约和故障处理的约束。

01

## 单一所有权

TaskTreeStore 是唯一运行时树；  
TaskTreeKernel 是唯一写入者。

02

## 类型化边界

分解器、系统操作、物理技能、修复器  
各有专属上下文。

03

## 任务与世界分层

每个任务新建树；物理世界、后端和事务账本持续复用。

04

## 工件不可变

计划携带快照、依赖版本和模型指纹；  
重规划发布新工件。

05

## 观测驱动闭环

动作后必采样；验证只读  
WorldSnapshot，不相信隐式内部状态  
。

06

## 失败显式递归

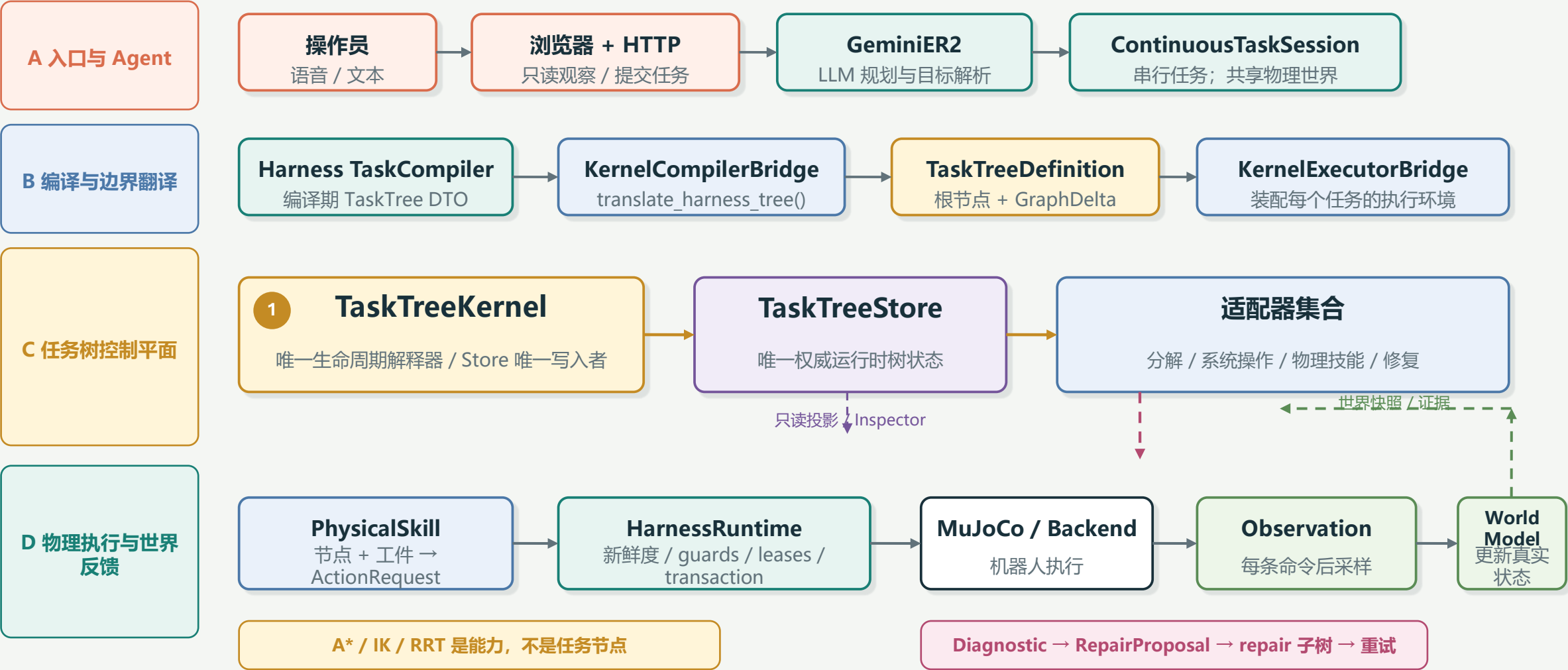
Diagnostic → RepairProposal →  
repair 子树 → 原节点重试。

**设计目标：把不可控的隐式行为，变成可审计的显式状态。**

因此，算法可以替换，机器人可以替换，任务树的控制逻辑和安全边界不需要被一起重写。

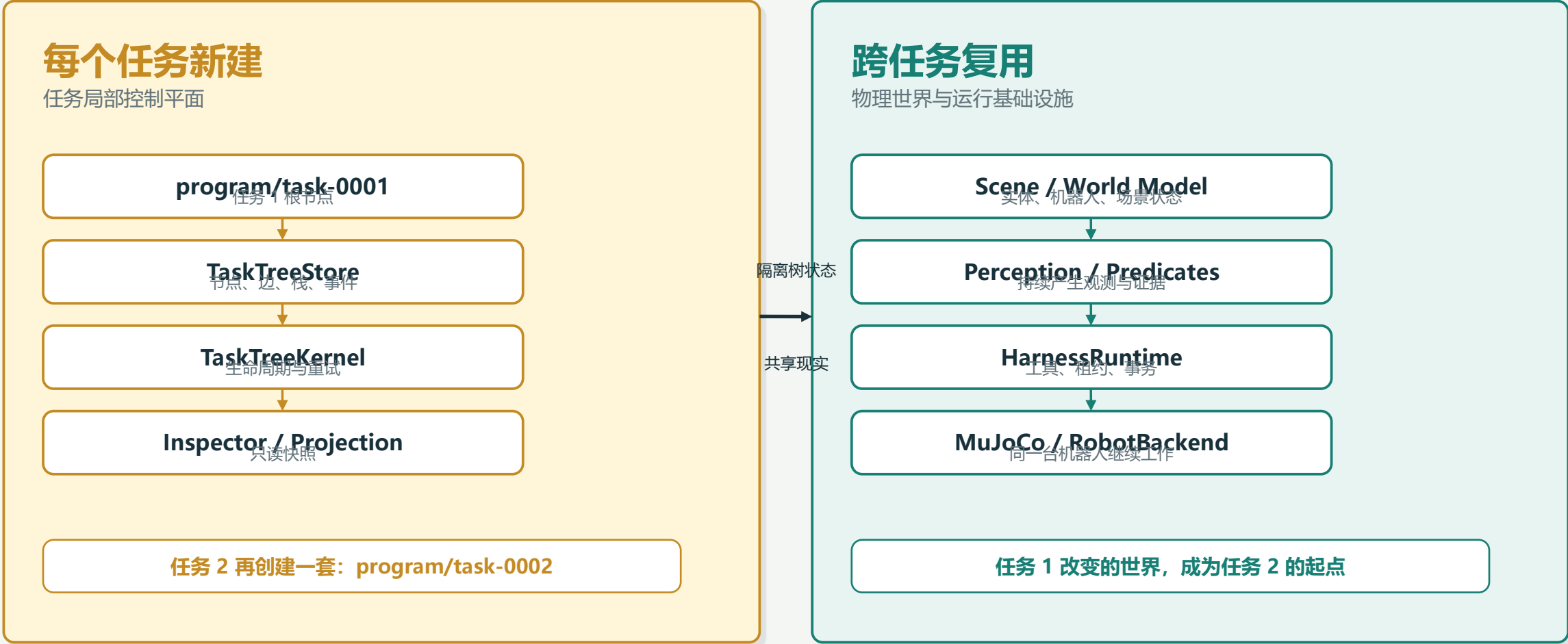
# 四层结构：入口、编译、任务树内核、物理世界

读图方向从左到右、从上到下；实线是主执行链，虚线是状态/证据反馈，粉色是修复回路。



# 最关键的工程取舍：树隔离，世界连续

如果把多个任务的节点混在一棵树里，审计和恢复会变复杂；如果每次连物理世界一起重置，又失去连续操作能力。



管理层语言：任务可以审计；现场不会被重置。

# TaskTreeKernel 是整个系统的执行控制平面

它不负责发明路径，也不负责控制电机；它负责决定哪个节点现在可以做、做完如何确认、失败如何收敛。

## 一个节点的生命周期



## Kernel 负责什么？

- 维护执行栈、节点运行态和事件日志
- 校验 GraphDelta：不越界、不成环、可达
- 统一处理重试、暂停、取消、预算和对账
- 决定何时调用分解器、系统操作、物理技能或修复器

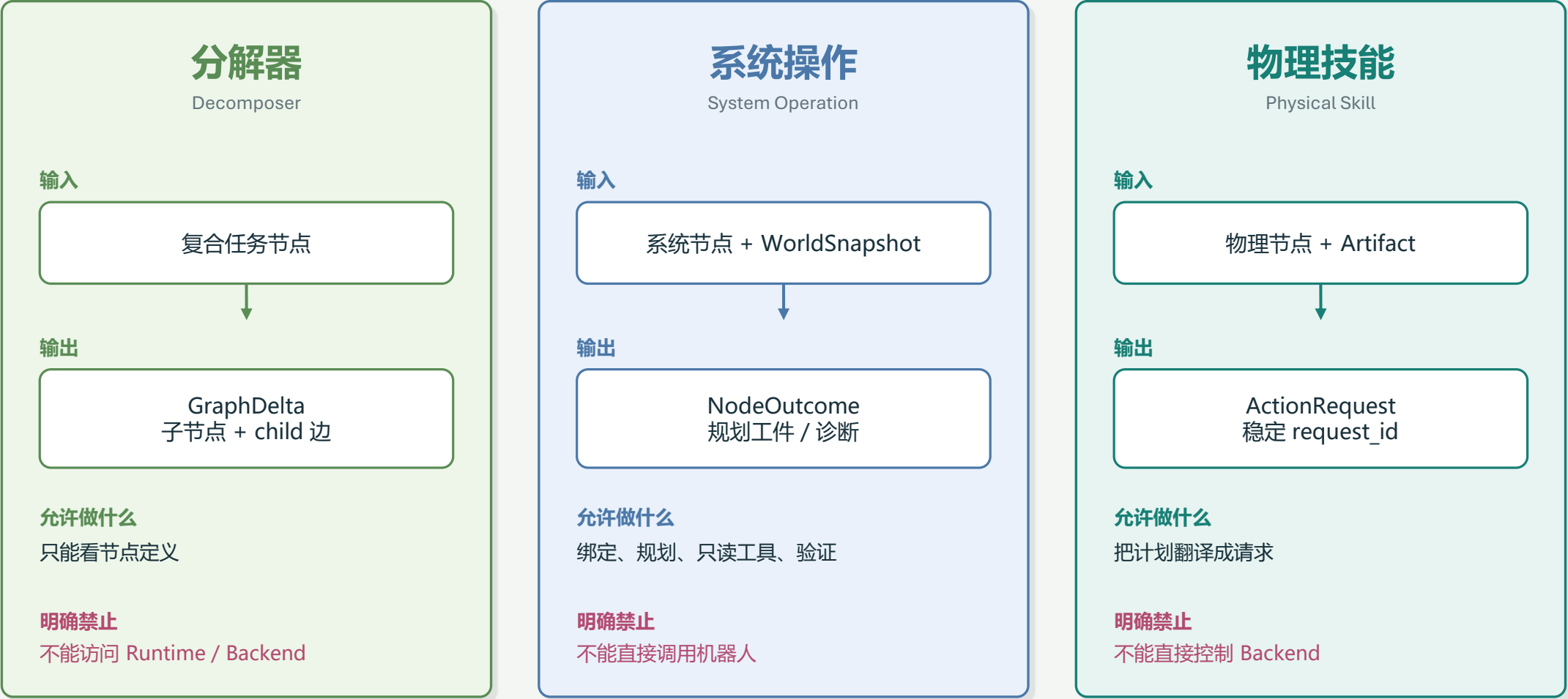
## Kernel 不负责什么？

- 不把 A\*、IK、RRT 变成任务节点
- 不直接读取机器人后端内部状态
- 不允许 LLM 或适配器直接修改 Store

唯一写入者 = Kernel ; 唯一权威状态 = Store

# 三类节点，三种权限，避免“万能 Agent 模块”

职责分离让扩展点清楚：新增任务不必改物理边界；更换机器人不必重写任务树。



特别重要：A\* / IK / RRT 属于 capabilities 的内部算法，不是任务树节点；算法迭代不会污染任务语义。

# 例 1：把红色杯子放进投放区，任务树如何展开？

编译器先只生成一个根节点；Kernel 运行到复合节点时，按需展开子树。

用户指令

“把红色杯子放进投放区”

系统先把自然语言变成两个确定的空间选择：

object\_selector → cup-red

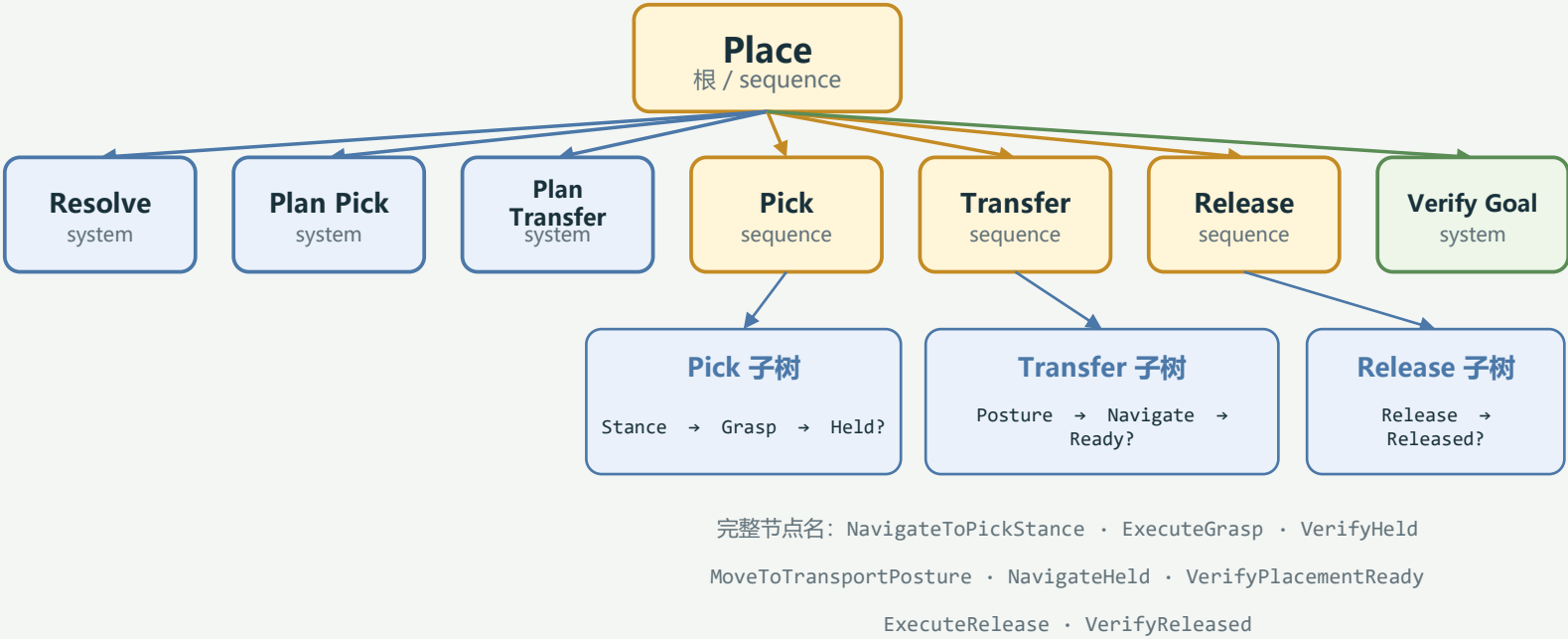
destination\_selector → drop-zone

根节点：Place

operation\_kind = decomposer

control\_kind = sequence

运行时任务树（成功后）



图中读法：黄色是复合控制节点，蓝色是系统/物理叶子；算法名称没有出现在树里。

16  
最终节点数

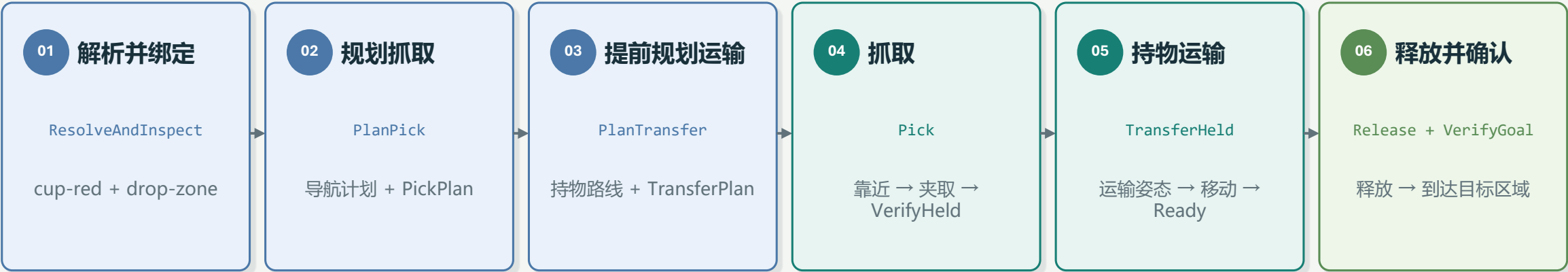
15  
child 边

0  
repair 边

成功  
根节点结果

# 正常放置不是一次调用，而是 5 次物理事务 + 多次系统验证

下图把系统动作和物理动作分开：系统节点负责决策与证据，物理节点负责请求与执行。



## 物理事务边界



真实运行结果：16 个节点、127 个事件、5 次物理事务，最终 world:7。

# 规划结果不是散落在内存里的变量，而是可追溯的不可变工件

工件连接了“谁生成、基于哪个世界、依赖什么、谁消费、何时失效”这几件事。

## 工件依赖链



### 每个工件携带的证据

- source snapshot: 例如 world:0
- 依赖实体 / 地图 / 机器人状态版本
- frame graph、机器人模型、碰撞模型版本
- payload transform hash (持物计划)
- 假设、随机种子和生产者 / 消费者契约

### 为什么要不可变？

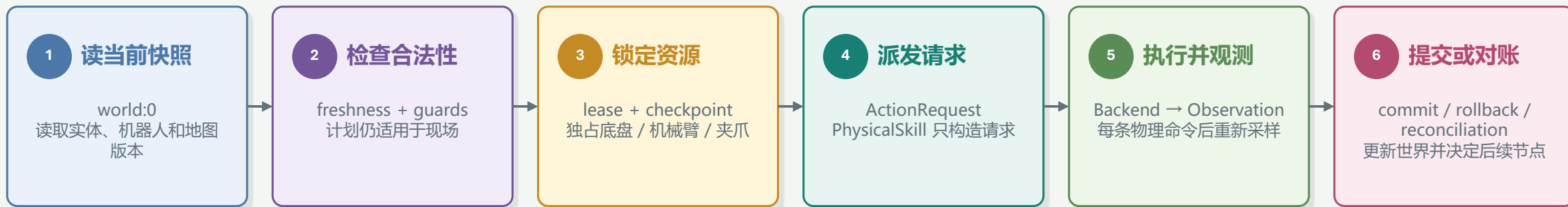
当世界改变时，不修改旧计划，而是：

- 1 **旧工件保留**      可审计：当时系统看到了什么
- 2 **新工件发布**      可恢复：重规划有明确来源
- 3 **别名指向新版本**      可执行：消费者拿到最新合法工件

计划失效是可计算的：比较声明的依赖版本与当前快照，而不是依赖一个模糊的 invalidated 标志。

# 一个物理动作，必须穿过六个阶段才算完成

以 NavigateHeld 为例：系统不是把计划直接交给机器人，而是先检查计划和现场，再执行、观测、提交或对账。



## 以 NavigateHeld 为例：计划如何变成事实

### 动作前

- TransferPlan 指向一个明确的 source snapshot
- 当前机器人、持物对象和路线依赖版本必须匹配
- 前置谓词成立，资源可独占，才能生成请求

### 动作后

Observation 写回 WorldModel; Verifier 根据新快照确认“仍然持有、已经到位、路线没有失真”。

### 三个必须坚持的判断

- API 返回成功，不等于任务后置条件成立
- 真实机器人不能假装回滚，只能观测后对账
- 动作后的世界版本，是下一步规划的输入

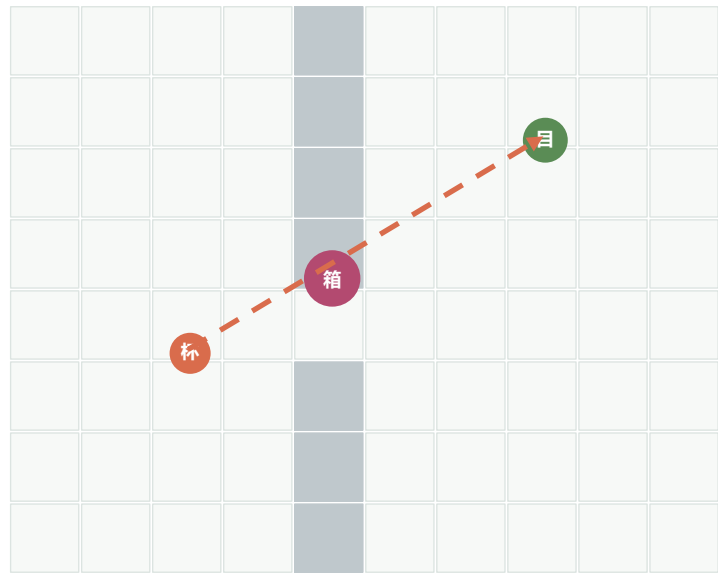
ActionRequest → Observation → Verify → Commit / Repair

**核心结论：任务成功 = 动作完成 + 观测有证据 + 后置谓词成立，而不是“命令发出去了”。**

# 例 2：路线被可移动箱子挡住，系统如何判断“该修什么”？

系统不会让 A\* 或控制器偷偷搬走障碍物；先证明阻挡者、可移动性和修复条件，再由任务树显式执行搬运。

## 参考场景 (10 × 8 网格)

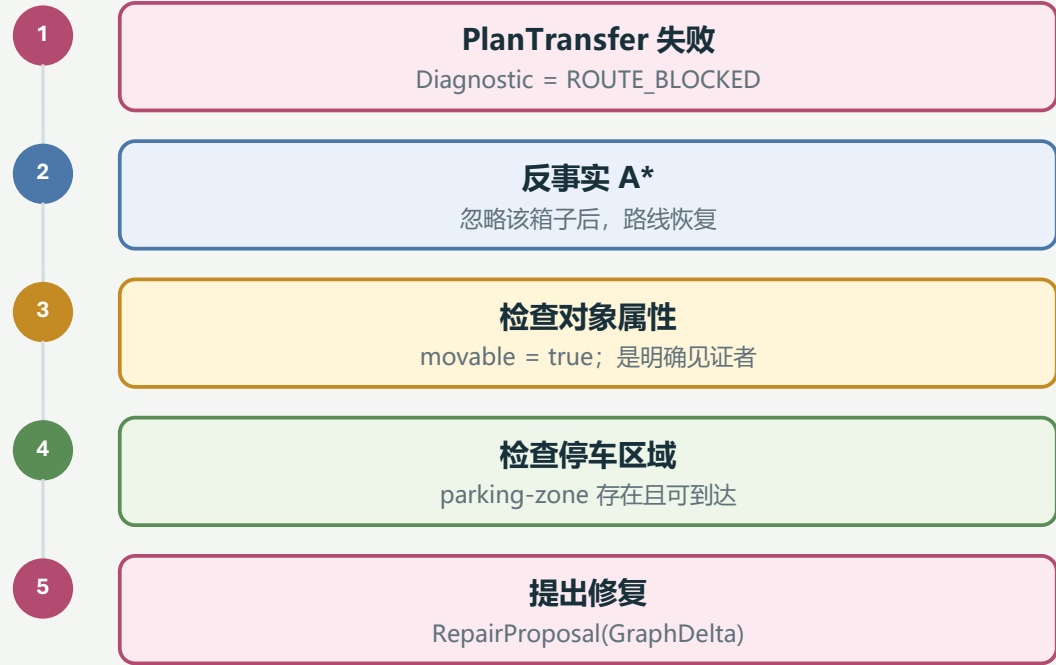


规划路线穿过箱子附近，连续碰撞检查拒绝该路线

阻挡物: movable-crate

修复区: parking-zone

## 修复判定链

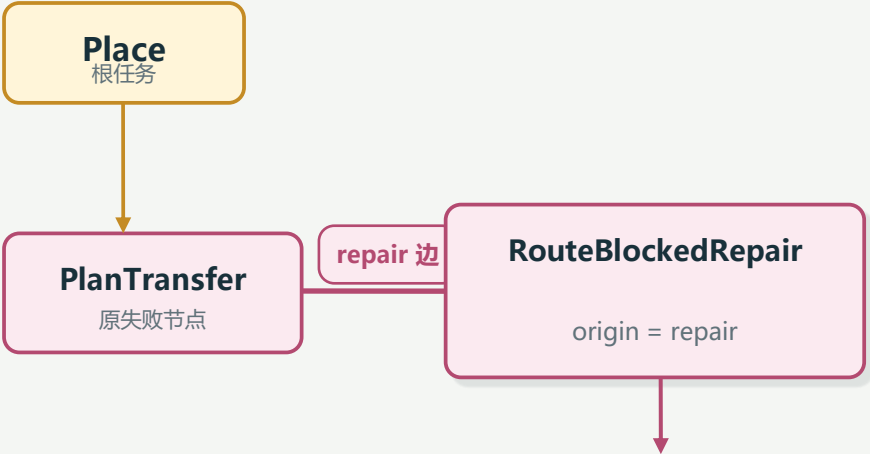


关键边界：修复动作本身也是任务，必须进入树、进入事件日志、进入物理事务。

# 修复不是旁路脚本，而是挂在失败节点上的一棵真实任务树

这让修复同样具备顺序、验证、物理事务、失败处理和递归深度限制。

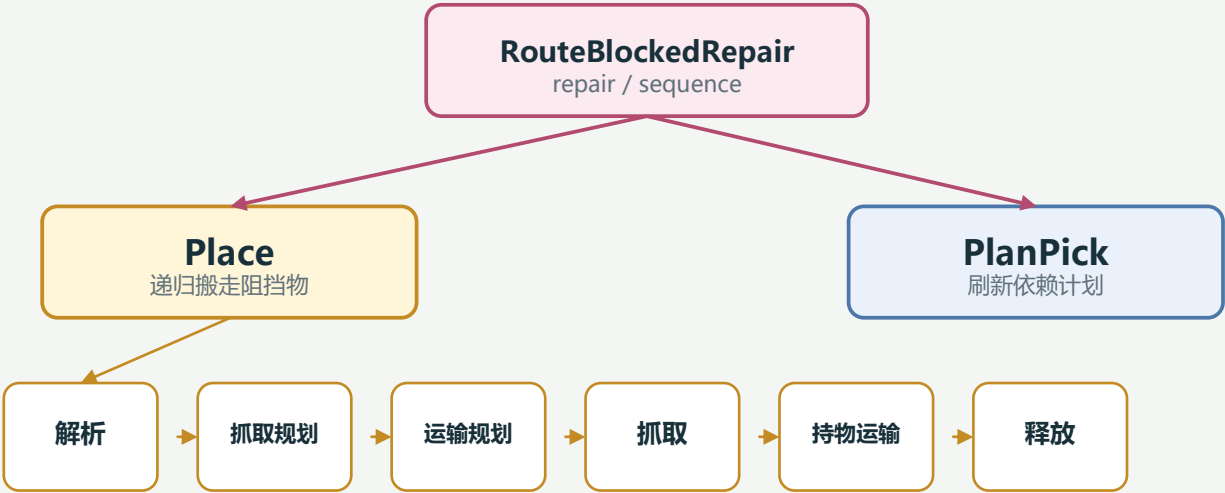
## 原任务树的关键位置



只允许一个修复入口，且修复子树必须从失败节点可达

内核先校验 GraphDelta：repair 边、入口节点、作用域、无环、可达。通过后才挂载。

## 修复子树展开



完整节点名：ResolveAndInspect → PlanPick → PlanTransfer  
Pick → TransferHeld → Release

Place(阻挡物, parking-zone) 自己也会展开为解析、规划、抓取、持物运输、释放和验证。

34

最终节点

1

repair 边

10

物理事务

275

事件

# 从路线失败到恢复成功，系统到底做了哪些事？

把故障处理拆成可讲解的 8 个动作，听众就能看到“递归”不是一句口号。



## 没有隐藏副作用

- A\* 只证明路线，不负责搬障碍物
- 搬箱子是 Place 任务，会进入物理事务
- 失败和修复都在同一棵可审计树里

## 重试的前提

- 新世界快照已被观测并进入 World Model
- 旧路线 / 计划按依赖关系重新检查新鲜度
- 只消费新的、契约匹配的工件

**最终结果：PlanTransfer 从第一次失败变成第二次成功；根任务仍然是同一个目标。**

# 连续操作：每条指令一棵新树，但现实世界不重置

这是产品化体验的核心：用户可以连续下达任务，系统保持现场状态，同时保证每棵树独立可审计。

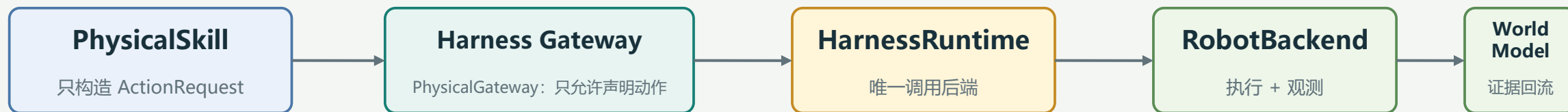


# 每条物理动作都要穿过一组明确的闸门

安全不依赖模型“自觉”，而是由工件契约、快照、资源、事务和观测共同约束。



## 物理边界的所有权



路线安全还要求连续验证：A\* 结果、实际起点/终点、结构化障碍物、机器人 footprint 和执行时重验证使用同一套策略。

# 这套架构带来的不是“多几个模块”，而是四种可量化能力

用管理层语言讲价值，用 CTO 语言讲机制；下面每项都能在源码和案例中找到对应证据。

可靠性

## 失败可收敛

路线阻塞会进入 repair 子树，修复后重试原目标，而不是靠模型随机重试。

案例二：34 节点 / 1 repair 边 / 成功

解释性

## 每一步有证据

树、事件、工件、快照和事务记录让动作前因后果可复盘。

案例一：127 个事件可追踪

扩展性

## 替换局部实现

新增任务加 decomposer；新增机器人实现 Backend；A\* / IK / RRT 可独立演进。

任务语义与算法解耦

安全性

## 物理动作有边界

只有 Runtime 能调用后端，工件、guards、租约和对账共同限制副作用。

动作后必观测

连续性

## 现场不重置

每个任务新树，World/Runtime/Backend 跨任务复用，适合多步协作。

task-0001 → task-0002

工程效率

## 问题定位更快

故障落在具体节点、具体工件和具体世界版本上，减少“整条链路猜测”。

局部重规划而非全量重做

管理层可见结果：风险可控、过程可解释、故障可恢复；技术团队可见结果：边界清晰、替换点明确、测试可落地。

# 参考实现已经把替换点留出来，下一步是把边界变成生产能力

当前仓库是 reference modular monolith；它验证的是架构契约，不等于所有生产基础设施已经完成。

## 当前参考实现

- TaskTreeStore：内存权威状态，JSON 用于检查
- 确定性平面双连杆模拟器 + 保守碰撞模型
- 本地单操作员 Web 控制台，默认 loopback
- 完整测试覆盖内核、工件、物理边界和修复
- GeminiER2 适配层复用 Harness 的世界与工具

架构验证重点：契约、所有权、恢复

## 生产化优先级

|    |           |                                                        |
|----|-----------|--------------------------------------------------------|
| P0 | 持久化 Store | SQLite/PostgreSQL/event log；树、栈、事件、工件同一事务边界            |
| P0 | 真实机器人适配   | 保留 ActionRequest、Observation、guard 和 reconciliation 契约 |
| P1 | 安全与运维     | 认证、多操作员、指标、分布式追踪、告警和回放                                 |
| P1 | 评测与策略     | 固定条件评测、故障注入、路线策略版本治理                                   |
| P2 | 规模化编排     | 任务模板、能力注册、资源池和多机器人协调                                   |

决策点：先选一个可观测、可回放、可注入故障的机器人任务作为生产试点。

## 核心判断

让 Agent 进入真实世界，关键不是让模型更“会说”，而是让执行过程更“可控”。

TaskTreeKernel

执行控制平面

统一解释任务树、状态和恢复

Harness

物理世界适配层

承载感知、工具、事务和机器人

Repair

一等公民

把失败变成可递归的工作

## 下一步建议

确定试点任务 → 接入持久化 Store → 接入真实机器人 → 建立故障注入与回放评测